Will They Solve It? Predicting Problem Solving Progress From Think Aloud Search Graphs
ECSE552 Project Proposal Group 6: Adnaan, Cher, Evans, Manuel

**Background**

Application: Tutors must decide when to step in while someone solves a problem. Help too early cuts learning short [1], while help too late leaves the learner stuck [2]. We study this using the game of 24 (combine 4 numbers from 1 to 13 and +, −, ×, ÷ to make 24), a standard reasoning task for people and language models [3, 4].

Problem: After each step, predict whether and how soon the person will solve the puzzle. Elapsed time or error counts ignore how the person searches, which think-aloud protocols expose [5, 6]. Wurgaft et al. described these graphs but did not forecast outcomes from them. Earlier work located where learners get stuck only after the fact [7]. To our knowledge, no model predicts this during the attempt, and if the forecast is accurate, ablations show which search behaviours predict success.

Constraints:

- Causal Input: at step t, the model sees only steps 1 to t.
- Censored outcomes: 46.5% of attempts time out unsolved; the model must learn from them, not drop them.
- Variable, unordered graphs: graphs differ in size and have no natural node order, so the architecture must be permutation-invariant.
- Calibration: predicted chances must match observed rates [8].
- Generalisation: accuracy must hold for unseen puzzles, people, and graphs from a different coder.

**Aims/Goals**

Measurable goals:

1. Beat every baseline on concordance index, with a paired 95% confidence interval above zero.
2. Keep expected calibration error below 0.05.
3. Lose at most 0.05 concordance versus a random split on unseen puzzles, participants or coders.

Scope:

In scope: the authors' coded Game of 24 graphs [3] and offline forecasts of time to solution.
Out of scope: transcription, building graphs, live tutoring, and seconds-level timing (no per-step timestamps exist).

Proposed Solution: A graph-sequence survival network.

- Graph encoder: a graph isomorphism network [9, 10] embeds the search graph after each step. Nodes are states (the numbers left); edges are operations tried, with backtracks marked. Node features include visit count, depth and whether 24 is still reachable, from an exact solver.
- Temporal encoder: a GRU [11] runs over the step embeddings to track revisits, dead ends and backtracking.
- Survival head: a discrete-time hazard layer [12] outputs the chance of solving at each of the next 10 steps, plus a later bin (86% of attempts have at most 10 operations; 5 and 15 tested). Timed-out attempts enter the loss as censored. A tutor can hint when the forecast drops.

**Methodology**

Data and Cleaning: We use the released think-aloud recordings of 640 participants, ten Game of 24 puzzles each, transcribed by Whisper and coded into a graph by Claude 3.5 Sonnet [3,13]. We apply the authors' exclusions (practice trials, irrelevant speech, 99 low-relevance participants: attempts over 181s unsolved) and drop graphs that fail to parse.

Implementation: We parse each attempt with the authors' GraphBuilder [13] into one graph snapshot per step, cut before any step that reaches 24 (a step is one operation tried; backtracks & subgoals update the graph but are not steps). Solved attempts are labelled with the step reaching 24; unsolved ones are right-censored: we know only that no solve happened by their last step, and loss uses only that. Models are built in PyTorch Geometric.

Evaluation:

- Validation and testing: five-fold cross-validation grouped by puzzle (unseen puzzles), repeated grouped by participant (unseen people). Hyperparameters are tuned on inner validation folds; test folds are scored once with the same folds for all alternatives.

- Metrics (test folds): concordance index [14], AUROC for solving within the next 3 steps (median solved length), expected calibration error [8] for that same event, and accuracy against steps seen.
- Robustness: train on Claude-coded graphs generated from original transcripts; test on DeepSeek-V3- and Llama-4-coded graphs and the human-coded subset.
- Reporting: mean ± SD over three seeds, paired bootstrap intervals, and ablations removing the graph encoder, and each feature group (backtracks, revisits, reachability), GRU and censoring.

Alternatives:

1. Puzzle only: solver-based difficulty (number of solutions), ignoring the attempt.
2. Trap-state lookup: mean steps to solution from each state in training data [7].
3. Hand-crafted features: Simple regression - Cox [15] and logistics regression on steps, backtracks, branching and reachability; gradient boosting as a stronger non-deep model.
4. Sequence only: a GRU over the step list without graph structure, as in deep knowledge tracing [16].
5. Language model: an LLM reads the transcript so far (rebuilt from each coded step's quote) and gives a probability(comparison only).

Our proposed solution could be superior because it combines graph structure, step order and censored outcomes.

**Feasibility and resources**

- Data: 4,947 human trials, 541 participants, 199 combinations after exclusions [3, 13]; 2,647 solved (53.5%) and 2,300 unsolved (46.5%) within the deadline.
- Scale: 30,105 operations, 15,527 backtracks; median 5 operations per attempt.
- Signal: Solved attempts take a median of 3 operations, unsolved 8; puzzle solve rates range from 0% to 100%.
- Compute: We expect training runs of minutes on one GPU (Digital Research Alliance of Canada).
- Timeline: weeks 1-2 snapshots, 3-5 model and tuning, 6-7 evaluation, ablation, robustness

**Risk analysis**

- Graphs coded from the full transcript leak the answer -> Cut each sequence before the step that reaches 24; never use the authors' reached-goal feature
- Puzzle difficulty explains most of the outcome -> Puzzle-grouped splits and the puzzle-only baseline; report what the attempt adds
- Machine-coded graphs are noisy (moderate agreement with humans [3]) -> Cross-coder and human-coded tests; report the gap
- A solved attempt's last step was never spoken -> Place the solve after the last observed step; check sensitivity
- Short attempts (median 5 steps) -> Small models, dropout, early stopping; 30,105 step-level examples
- The network does not beat the baselines -> Ablations still show what structure and censoring add; report the negative result

**Role of group members**

| Member | Tasks |
|---|---|
| Cher | Graph snapshots per step (GraphBuilder); puzzle and participant grouped splits; DeepSeek and Llama-4 versions. |
| Manuel | Graph encoder, GRU, and survival head, training and tuning. |
| Evans | Baselines: puzzle-only / language model, trap-state lookup, Cox and gradient boosted models. |
| Adnaan | Evaluation: metrics, calibration, robustness tests, other necessary plots. |

**Citations**


1. Koedinger, K. R., & Aleven, V. (2007). Exploring the assistance dilemma in experiments with Cognitive Tutors. *Educational Psychology Review*, 19(3), 239-264.
2. Beck, J. E., & Gong, Y. (2013). Wheel-spinning: Students who fail to master a skill. *Artificial Intelligence in Education (AIED 2013)*, LNCS 7926, 431-440.
3. Wurgaft, D., Prystawski, B., Gandhi, K., Zhang, C. E., Tenenbaum, J. B., & Goodman, N. D. (2025). Scaling up the think-aloud method. *Proceedings of the Annual Meeting of the Cognitive Science Society*. arXiv:2505.23931.
4. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS 36*.
5. Newell, A., & Simon, H. A. (1972). *Human problem solving*. Prentice-Hall.
6. Ericsson, K. A., & Simon, H. A. (1993). *Protocol analysis: Verbal reports as data* (rev. ed.). MIT Press.
7. Supianto, A. A., Hayashi, Y., & Hirashima, T. (2016). Visualizations of problem-posing activity sequences toward modeling the thinking process. *Research and Practice in Technology Enhanced Learning*, 11, 14.
8. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML*.
9. Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? *ICLR*.
10. Hamilton, W. L. (2020). *Graph representation learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning, 14(3). Morgan & Claypool.
11. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.
12. Gensheimer, M. F., & Narasimhan, B. (2019). A scalable discrete-time survival model for neural networks. *PeerJ*, 7, e6257.
13. Prystawski, B., et al. (2025). think-aloud-llms: code and data for "Scaling up the think-aloud method". GitHub repository, MIT licence.
14. Harrell, F. E., Lee, K. L., & Mark, D. B. (1996). Multivariable prognostic models: Issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors. *Statistics in Medicine* 15(4), 361-387.
15. Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187-220.
16. Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. *NeurIPS 28*.